

Tensor networks techniques for differential equations: Part IV

QTADE School 2026, Bilbao

Stefano Pisoni
Researcher, Quantum Algorithms
QRC, TII Abu Dhabi, UAE
PhD, TUHH

Raghavendra Peddinti
Researcher, Quantum Algorithms
QRC, TII Abu Dhabi, UAE

Recap

Part 1: Function representation using QTTs.

Part 2: QTT Linear algebra subroutines.

Part 3: The forward problem: solving PDEs (Navier-Stokes).

Part 4: The inverse problem: data-driven models.

Outline

The Inverse Problem

Dynamic Mode Decomposition

MPS-DMD (a bonus PDE solver)

SINDy, MANDy and more.

Classification of our tasks

Key scientific tasks: simulation

$$b = F(a)$$

a = set of input conditions

F = physical model of the system (usually a PDE)

b = resulting properties given F and a

Classification of our tasks

Key scientific tasks: inverse problems

$$b = F(a)$$

a = set of input conditions

F = physical model of the system (usually a PDE)

b = resulting properties given F and a

Classification of our tasks

Key scientific tasks: discovering unknown physics

$$b = F(a)$$

a = set of input conditions

F = physical model of the system (usually a PDE)

b = resulting properties given F and a

$$\text{given } \{(a_i, b_i)\}_{i=1}^M \longrightarrow \hat{F} = \arg \min_{G \in \mathcal{M}} \sum_i \|b_i - G(a_i)\|^2$$

Motivation

We solved PDEs. Why data-driven?
Measurements >>>> modelling.

Why not AI/ML?

Naively using deep learning for scientific tasks usually leads to:

- Lack of interpretability
- Poor generalisation
- Lots of training data required

Problem statement

- Given measurement data as a time series:

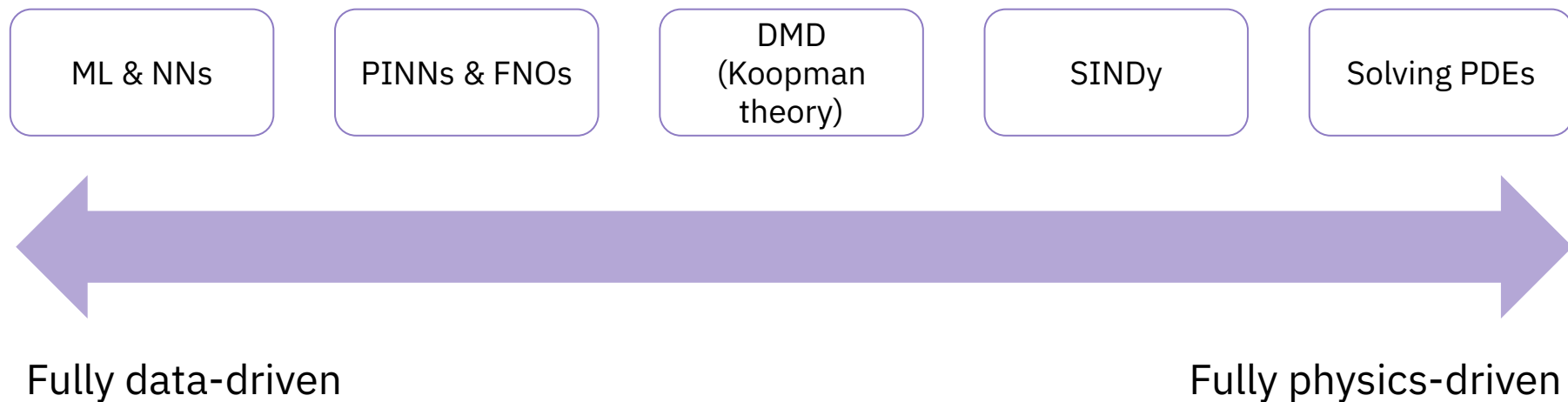
$$\mathcal{D} = \{\mathbf{u}_k\}_{k=0}^{N_t}, \quad \mathbf{u}_k = \mathbf{u}(t_k) \in \mathbb{R}^{N_x}, \quad t_k = k \Delta t$$

$$\frac{d\mathbf{u}}{dt} = \mathbf{f}(\mathbf{u}) \implies \mathbf{u}_{k+1} = \mathbf{F}(\mathbf{u}_k)$$

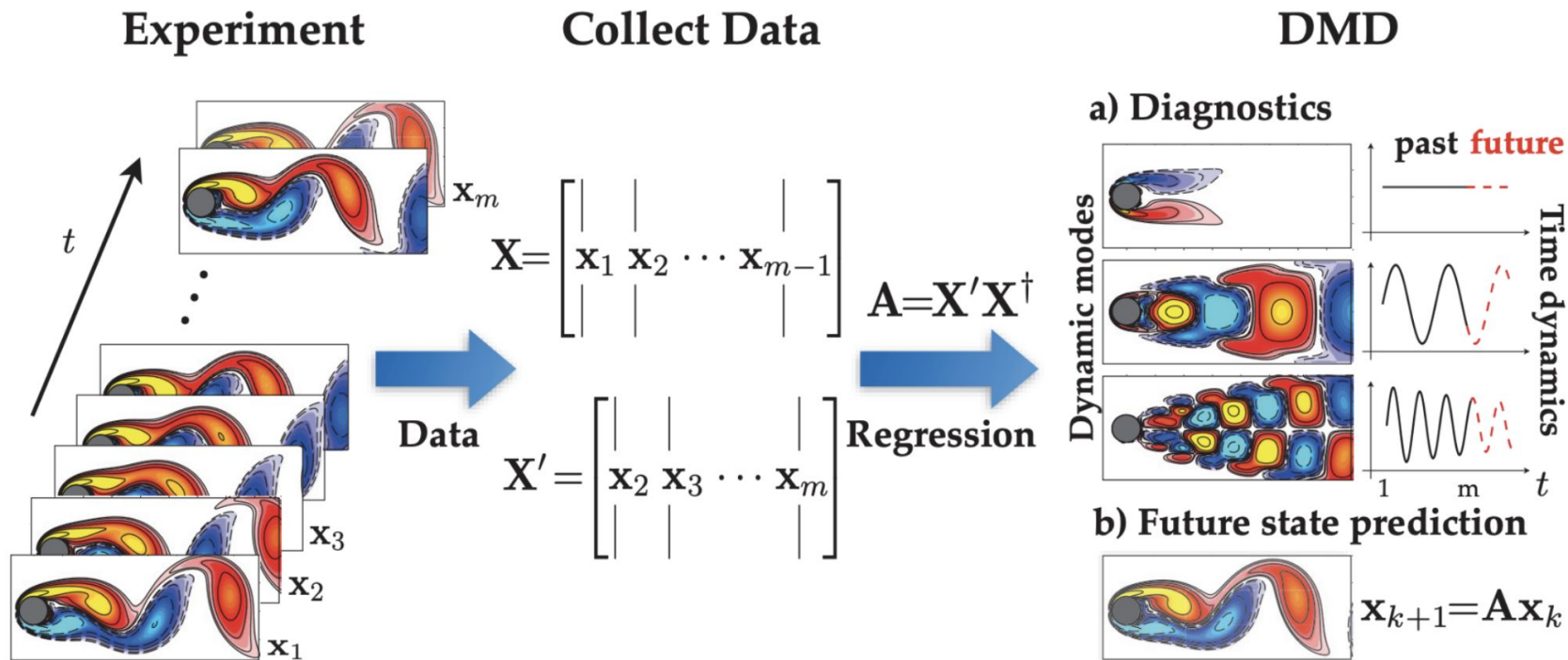
$$\hat{\mathbf{F}} = \arg \min_{\mathbf{G} \in \mathcal{M}} \sum_{k=0}^{N_t-1} \|\mathbf{u}_{k+1} - \mathbf{G}(\mathbf{u}_k)\|_2^2$$

- Prediction task:
 - Predict future states: $\hat{\mathbf{u}}_{N_t+k} = \hat{\mathbf{F}}^k(\mathbf{u}_{N_t})$
- Learning task:
 - Characterize the spectrum: $\hat{\mathbf{F}}(\mathbf{u}) = A\mathbf{u}, \quad A\phi_j = \lambda_j\phi_j$

The spectrum



Dynamic mode decomposition



Space-time MPS encoding

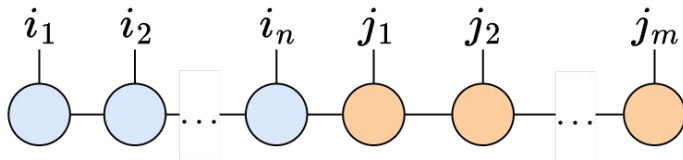
#1. Discretization

$$x \longleftrightarrow \{i_0, i_1, \dots, i_n\} \quad t \longleftrightarrow \{j_0, j_1, \dots, j_m\}$$

$$f(x, t) \longleftrightarrow f(i_0, \dots, i_n, j_0, \dots, j_m)$$

#2. MPS encoding

$$f(x, t) \rightarrow A_1^{(i_1)} A_2^{(i_2)} \dots A_n^{(i_n)} A_{n+1}^{(j_1)} A_{n+2}^{(j_2)} \dots A_{n+m}^{(j_m)}$$

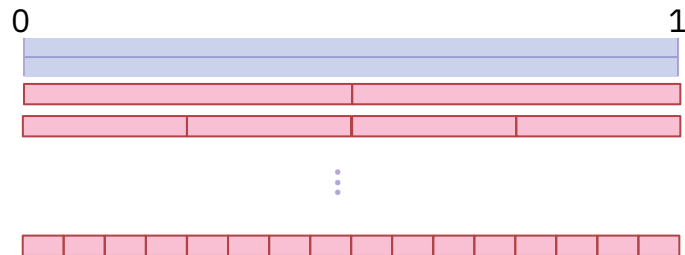


Space-time MPS encoding

#1. Discretization

$$x \longleftrightarrow \{i_0, i_1, \dots, i_n\} \quad t \longleftrightarrow \{j_0, j_1, \dots, j_m\}$$

$$f(x, t) \longleftrightarrow f(i_0, \dots, i_n, j_0, \dots, j_m)$$



$$x = \sum_{k=1}^n i_k 2^{-k}, \quad i_k \in \{0, 1\}$$

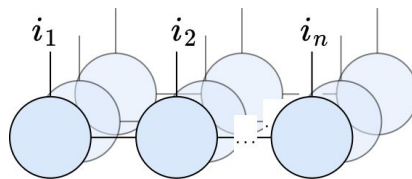
$$t = T \sum_{l=1}^m j_l 2^{-l}$$

A bonus PDE solver!

$$\partial_t u + F(u, \partial_x u, \dots, \partial_x^q u) = g(x, t)$$

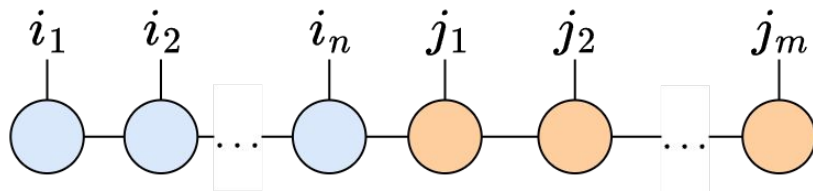
Time-stepping

$$\mathbf{u}_{t+1} = \mathbf{u}_t + \mathbb{F}(\mathbf{u}_t)\Delta t$$



Space-time methods

$$(\mathbb{I}_x \otimes \mathbb{D}_t + \mathbb{F}_x \otimes \mathbb{I}_t)\mathbf{u} = \text{b.c.} + \text{i.c.}$$



- Nonlinear F requires Newton iterations w/ space-time Jacobian

MPS-DMD

Snapshot matrices

$$X = [\mathbf{u}_0, \dots, \mathbf{u}_{N_t-1}]$$

$$X' = [\mathbf{u}_1, \dots, \mathbf{u}_{N_t}]$$

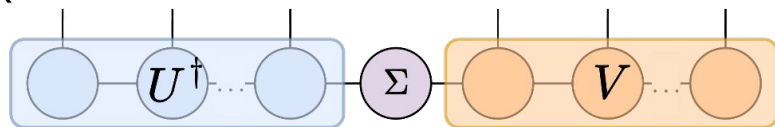
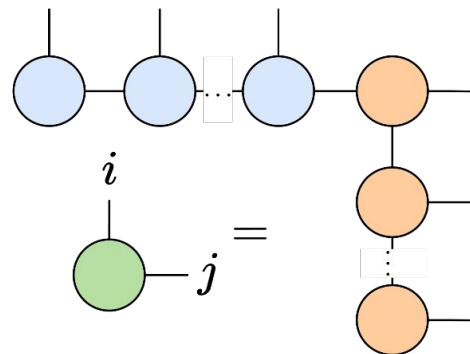
Goal: Find A s.t.

$$X' = AX$$

1. Proper orthogonal decomposition (POD) of X

$$X = U\Sigma V^\dagger$$

$$\Rightarrow A = X'X^+ = X'V\Sigma^{-1}U^\dagger$$



MPS-DMD

2. Project A onto the span of U

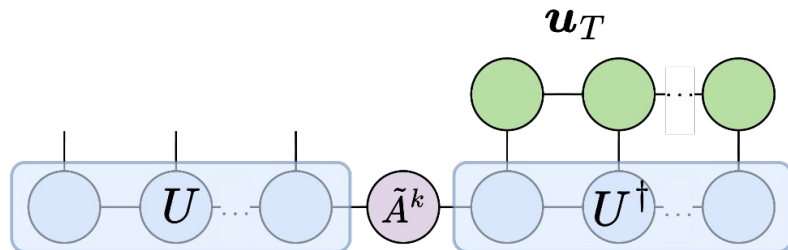
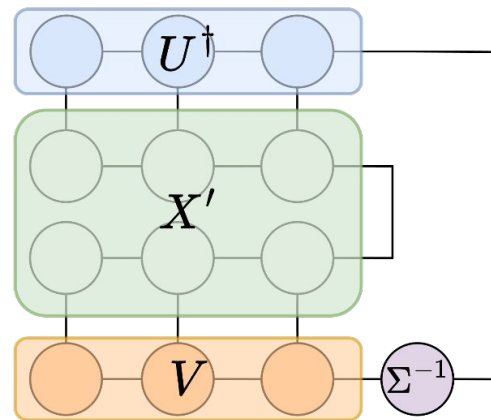
$$\begin{aligned}\tilde{A} &:= U^\dagger A U \\ &= U^\dagger X' V \Sigma^{-1}\end{aligned}$$

3. Eigen decomposition of A

$$\begin{aligned}\tilde{A} W &= W \Lambda \\ \implies A U W &= U W \Lambda\end{aligned}$$

4. Prediction

$$\begin{aligned}\mathbf{u}_{T+k} &= A^k \mathbf{u}_T \\ A^k &= U W \Lambda^k W^{-1} U^\dagger\end{aligned}$$



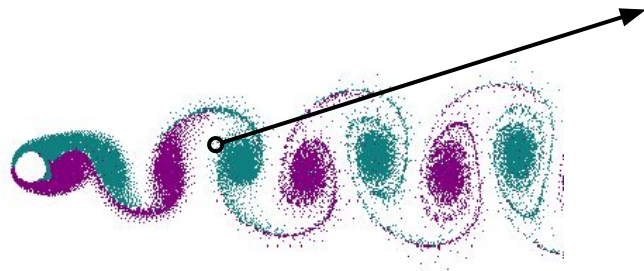
TTs for DMD

- Klus, Stefan, et al. (2018) "Tensor-based dynamic mode decomposition."
 - Original based using spatial compression
 - Temporal dimension left uncompressed, i.e. linear complexity.
- Środa, Maksymilian, et al. (2026) "Predictor-corrector method based on dynamic mode decomposition for tensor-train nonequilibrium Green's function calculations."
 - Independent work with the same complexity.

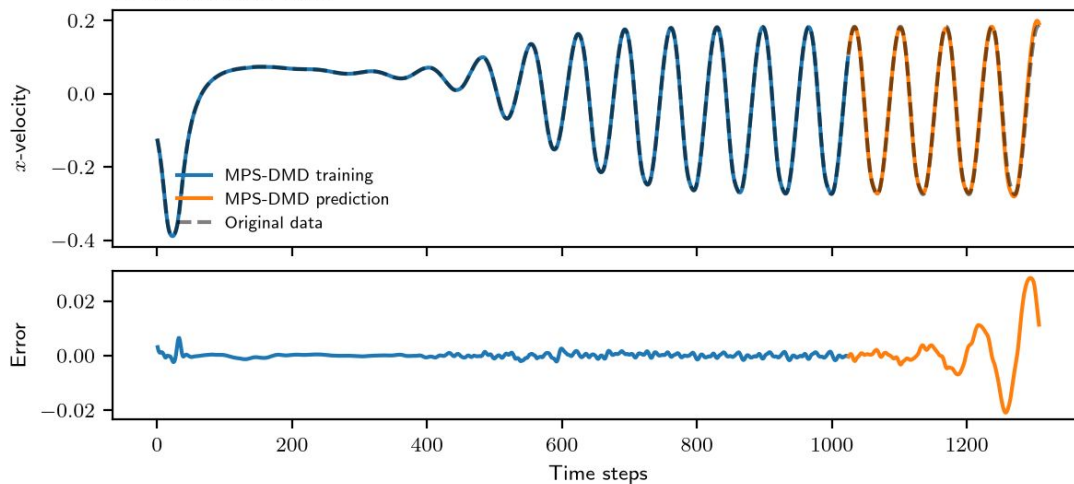
DMD: $\mathcal{O}(N_x N_t \min(N_x, N_t))$

MPS-DMD: $\mathcal{O}((n + m) \chi^3)$, $N_x = 2^n$, $N_t = 2^m$

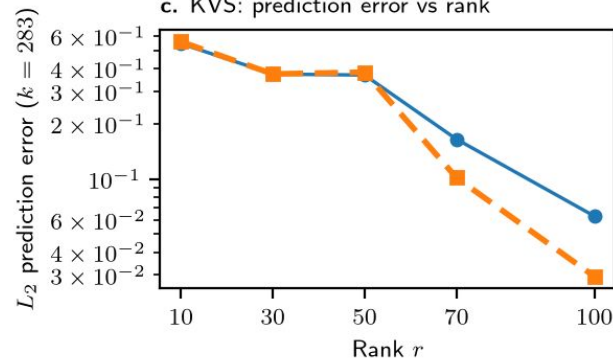
MPS-DMD results



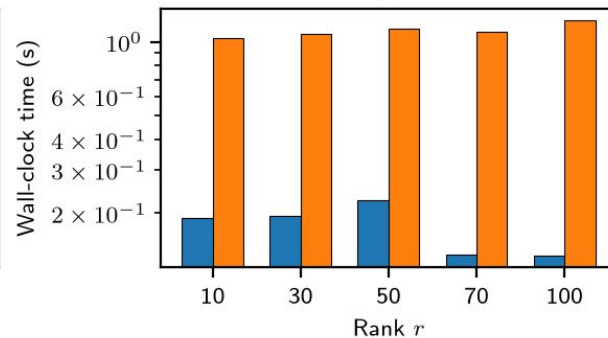
b. Kármán vortex street



c. KVS: prediction error vs rank



d. KVS: wall-clock by phase



SINDy

- Original dynamics: $\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t))$

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}^T(t_1) \\ \mathbf{x}^T(t_2) \\ \vdots \\ \mathbf{x}^T(t_m) \end{bmatrix} = \overbrace{\begin{bmatrix} x_1(t_1) & x_2(t_1) & \cdots & x_n(t_1) \\ x_1(t_2) & x_2(t_2) & \cdots & x_n(t_2) \\ \vdots & \vdots & \ddots & \vdots \\ x_1(t_m) & x_2(t_m) & \cdots & x_n(t_m) \end{bmatrix}}^{\text{state}} \underbrace{\quad}_{\text{time}}$$

- Data:

$$\dot{\mathbf{X}} = \begin{bmatrix} \dot{\mathbf{x}}^T(t_1) \\ \dot{\mathbf{x}}^T(t_2) \\ \vdots \\ \dot{\mathbf{x}}^T(t_m) \end{bmatrix} = \begin{bmatrix} \dot{x}_1(t_1) & \dot{x}_2(t_1) & \cdots & \dot{x}_n(t_1) \\ \dot{x}_1(t_2) & \dot{x}_2(t_2) & \cdots & \dot{x}_n(t_2) \\ \vdots & \vdots & \ddots & \vdots \\ \dot{x}_1(t_m) & \dot{x}_2(t_m) & \cdots & \dot{x}_n(t_m) \end{bmatrix}.$$

- Recovered dynamics: $\dot{\mathbf{X}} = \mathbf{\Theta}(\mathbf{X})\mathbf{\Xi}$

SINDy: Curse of dimensionality

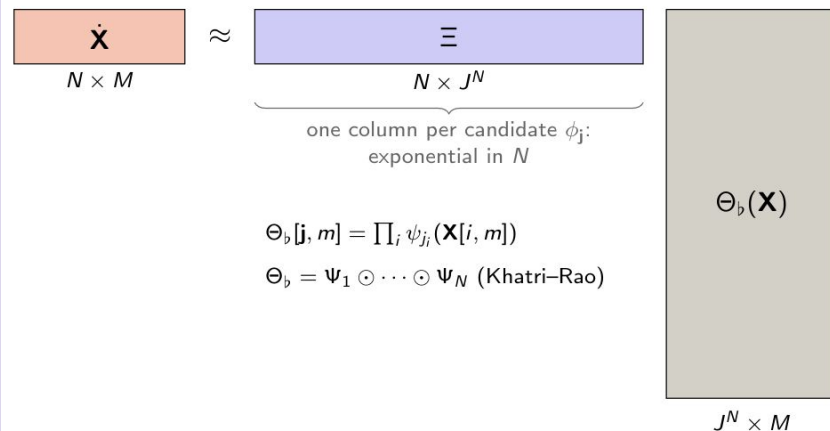
- Data: $\mathbf{X} \in \mathbb{R}^{N \times M}$, $\mathbf{X}[i, m] \approx x_i(t_m)$
 - M: no. of time samples
 - N: state dimension

$$\psi_1, \dots, \psi_J : \mathbb{R} \rightarrow \mathbb{R},$$

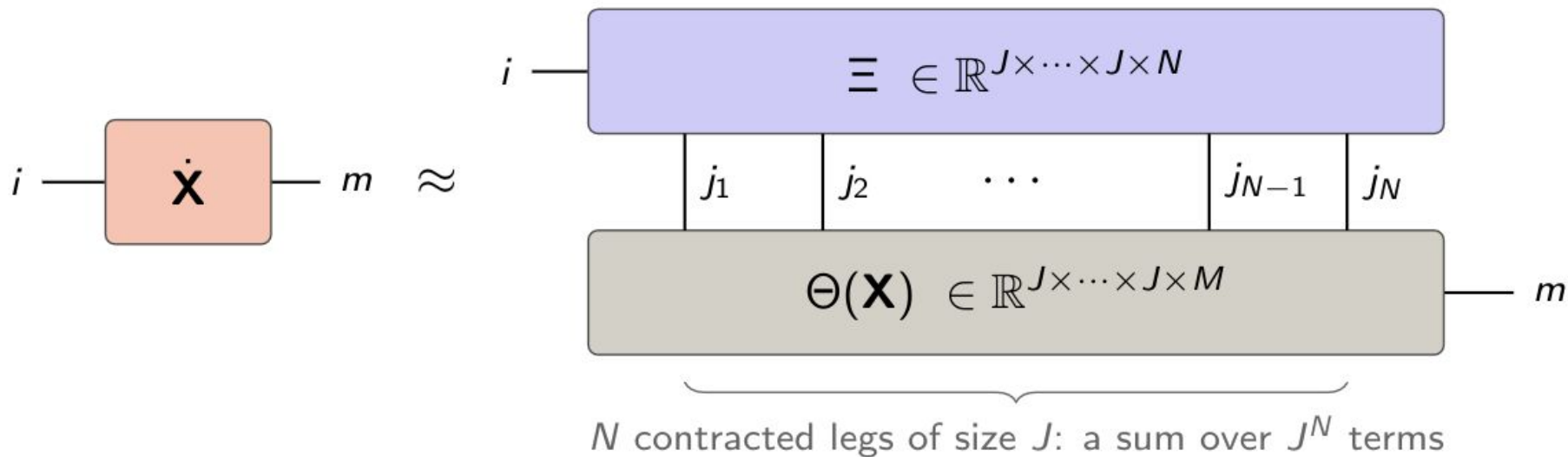
$$\phi_{\mathbf{j}}(\mathbf{x}) = \prod_{i=1}^N \psi_{j_i}(x_i), \quad \Theta[\mathbf{j}, m] = \phi_{\mathbf{j}}(\mathbf{X}[:, m]),$$

- Dictionary: Θ is of size $J^*J^*\dots J^*M$
 - (J multiplied N times)
- Coefficients: Ξ is of size $N^*J^*J^*\dots J$

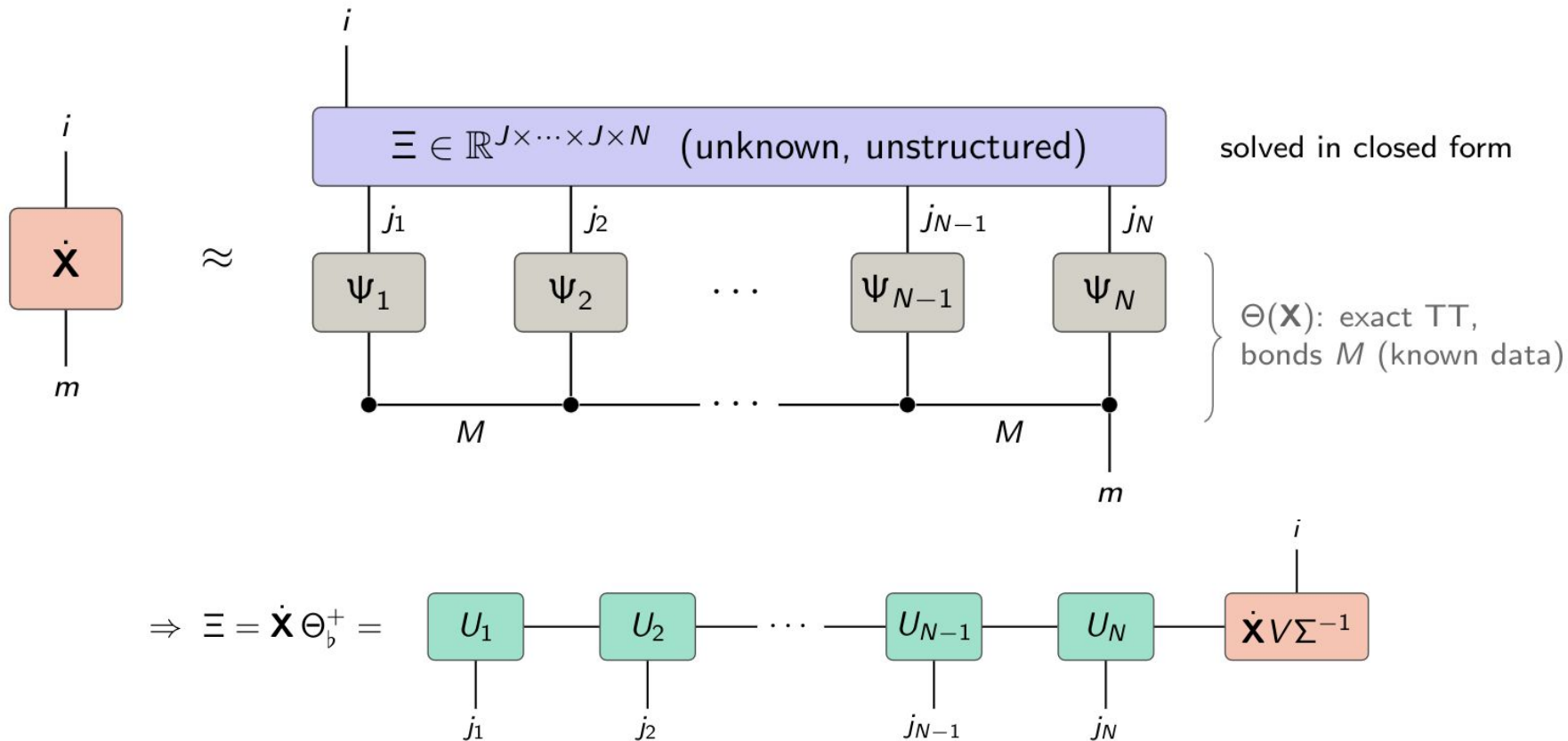
$$\min_{\Xi} \|\dot{\mathbf{X}} - \Xi \Theta_{\mathbf{b}}(\mathbf{X})\|_F^2 + \mathcal{R}(\Xi)$$



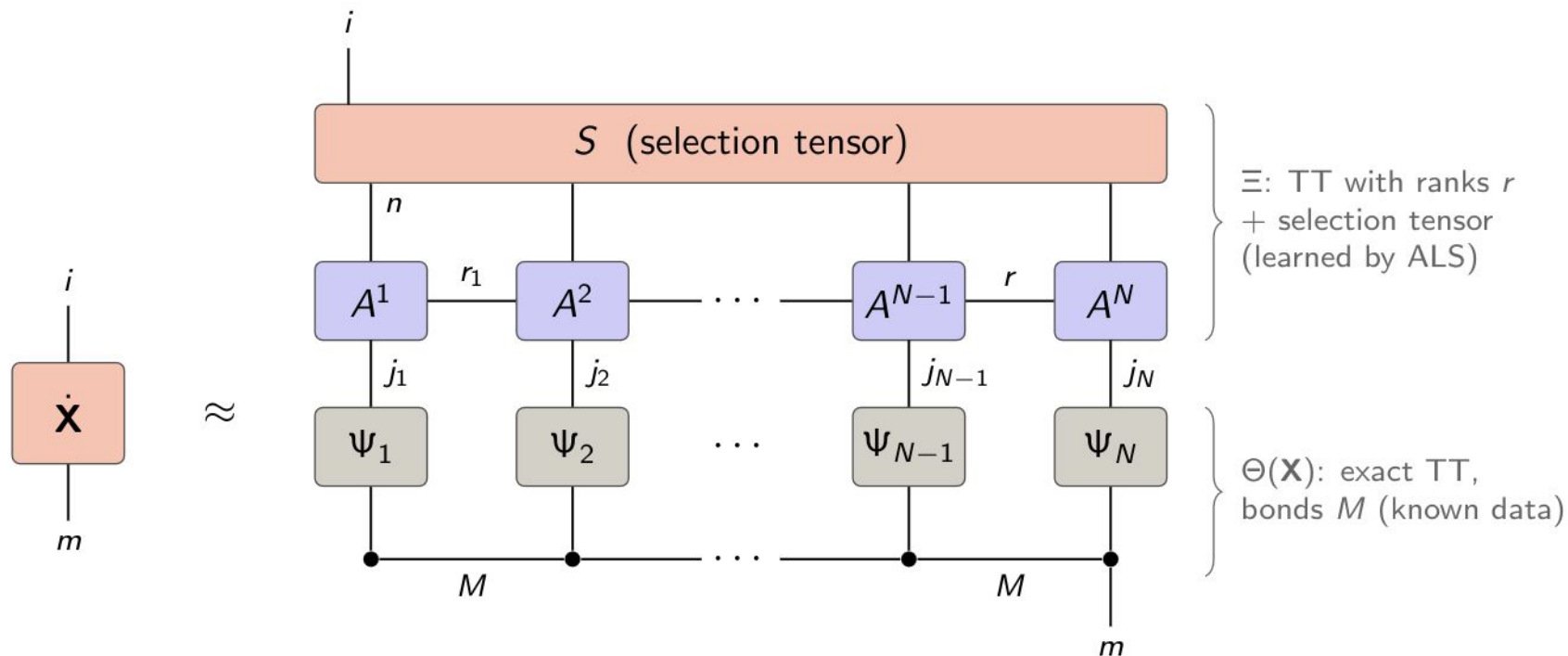
SINDy: Tensor form



MANDy and more.



MANDy and more.



More papers to read.

Tensor-Train Weak SINDy: Identifying High-Dimensional Nonlinear Dynamics

Will Houser*, Vanja Dukic*, and David M. Bortz*

Abstract. In recent years, weak-form methods have made significant advances in data-driven discovery of dynamical systems. However, in high-dimensional settings, current techniques can prove expensive in both computation and memory. In this work, we introduce TT-WSINDy, which combines techniques of the Multidimensional Approximation of Nonlinear Dynamics (MANDy) and Weak Sparse Identification of Nonlinear Dynamics (WSINDy) methods, implementing requisite computations in the tensor-train (TT) format. We demonstrate that this method is able to search an exponentially-growing space of candidate functions – performing weak-form transformation, regression, and sparsification – without suffering from the curse of dimensionality.

arXiv:2609.09434v1 [cs.LG] 8 Sep 2026

An interesting application

1) atmospheric chemistry

simulate w/o aviation emissions

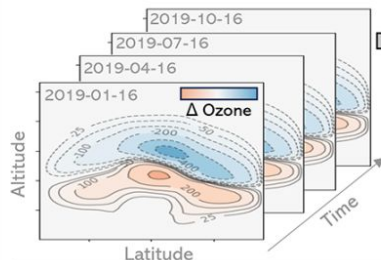


simulate w/ aviation emissions



2) preprocessing data

subtract perturbed from reference atmosphere to isolate differences

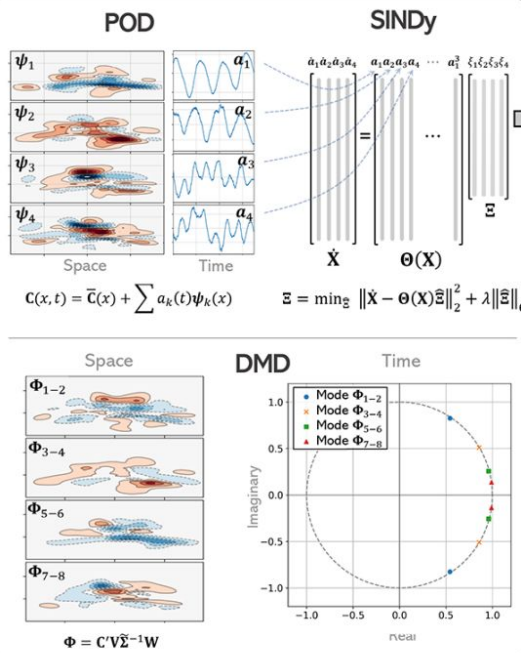


organise data in snapshot matrices

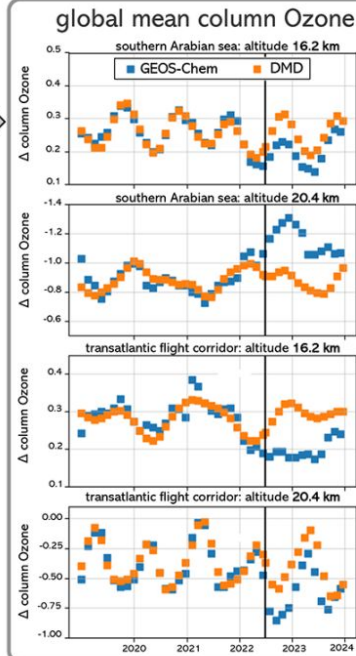
$$C = \begin{bmatrix} | & | & & | \\ c(t_1) & c(t_2) & \cdots & c(t_{m-1}) \\ | & | & & | \end{bmatrix}$$

$$C' = \begin{bmatrix} | & | & & | \\ c(t_2) & c(t_3) & \cdots & c(t_m) \\ | & | & & | \end{bmatrix}$$

3) model discovery

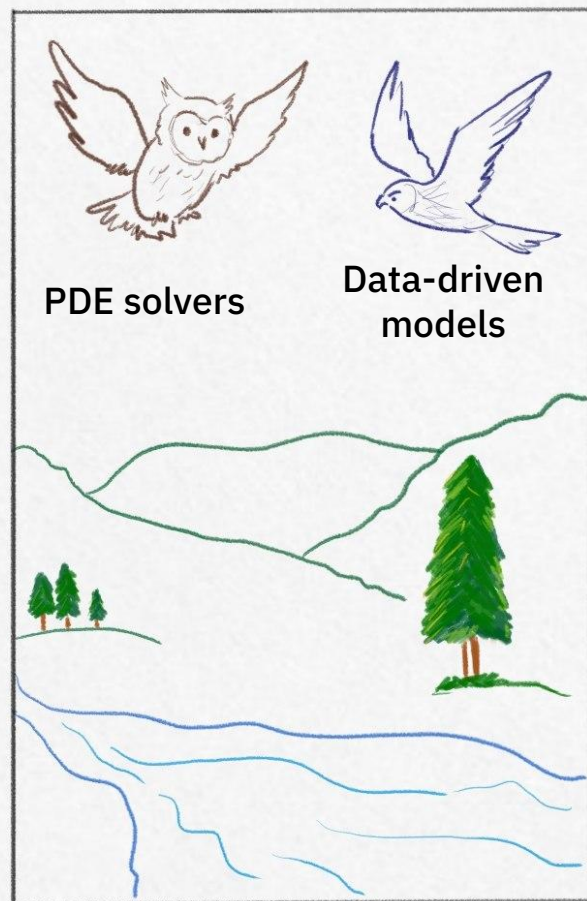


4) reduced order models

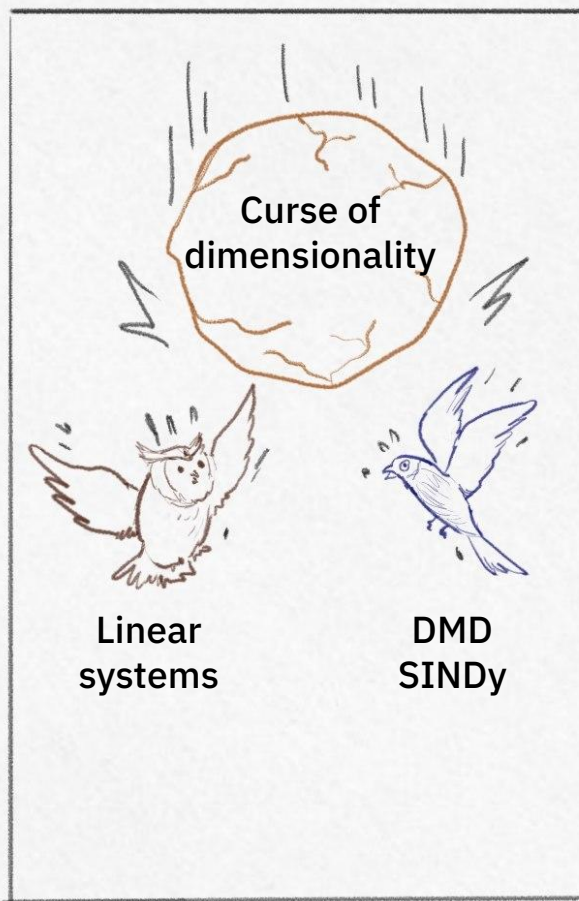


Thank you. Questions?

The land of Dynamical systems



One stone, two birds



Restoring hope

